Discover Internet of Things

RESEARCH Open Access

Development of an intelligent intrusion detection system for IoT networks using deep learning

Haozhe Zhang[1*]

*Correspondence:
Haozhe Zhang
zhanghaozhe202503@163.com
[1]Department of Computer Science, University of Sheffield, Sheffield S10 2TN, UK

Abstract

The Internet of Things (IoT) enables transformative applications; however, it faces significant security and privacy challenges. Due to the impracticality of securing individual devices, network-level security is preferred. However, attack diversity, device heterogeneity, and traditional security limitations necessitate advanced data analysis. Researchers increasingly use deep learning, which excels in handling large-scale data, to develop robust intrusion detection systems. This study investigates the security challenges of IoT, reviews existing intrusion detection systems, and explores the application of deep learning for intrusion detection in IoT networks. A total of 21 neural network models, along with an ensemble learning model, were designed and trained using the BoT-IoT dataset. Performance was evaluated using standard classification metrics including accuracy, precision, recall, and F1-score. The ensemble learning model demonstrated the highest performance, achieving 99.985% accuracy, 99.28% precision, 99.80% recall, and 99.54% F1-score. Compared to previous studies, the proposed model improves accuracy, recall, and F1-score by 0.005%, 0.55%, and 0.84%, respectively. These results highlight the effectiveness of deep learning-based approaches in enhancing IoT network security.

Article Highlights

1.High accuracy in threat detection: The ensemble learning model achieved near-perfect accuracy (99.985%) in identifying IoT network attacks, outperforming previous methods.
2.Effective use of deep learning: Combining multiple neural network models improved detection rates for diverse cyber threats in IoT environments.
3.Practical IoT security solution: The approach offers a scalable way to protect IoT networks without relying on individual device security.

Keywords Security, Internet of Things (IoT), Intrusion detection, Deep learning, Botnet

1 Introduction

The Internet of Things (IoT) connects smart devices, enabling automated data exchange and improving data collection, measurement, and awareness. With an estimated 30 billion connected devices by 2024, IoT is experiencing rapid growth; however, it remains

vulnerable to cyberattacks due to its reliance on internet connectivity [1–3]. Intrusion Detection Systems (IDS) help secure networks, but traditional methods struggle with resource limitations, specialized protocols, and lack of standardization. To address this, researchers are adopting machine learning and deep learning for intrusion detection in IoT. These techniques analyze large-scale data to detect threats effectively, enhancing security and enabling adaptive, efficient protection mechanisms for IoT networks. The Internet of Things (IoT) improves smart applications but faces security risks due to device vulnerabilities. Cyberattacks like data theft and network disruption exploit weak security. Limited computational power prevents strong protection mechanisms, making IoT an easy target. Secure, efficient solutions are essential to safeguard privacy and reliability in IoT networks [4, 5]. Securing individual IoT devices is impractical due to their vast numbers and heterogeneity, making network-level security essential. Traditional methods are ineffective against evolving threats, but AI-based approaches, particularly machine learning (ML) and deep learning (DL), enhance Intrusion Detection Systems (IDS). DL models excel at extracting complex patterns, enabling early threat detection and improved cybersecurity [6, 7].

The IoT introduces significant cybersecurity threats, with notable attacks like Mirai in 2016, which infected millions of devices for DDoS attacks [8]. Cyber-Physical Systems (CPS), benefiting from IoT, integrate sensors and devices for enhanced functionality in areas like healthcare, transportation, and smart energy management. However, these systems'security vulnerabilities in critical infrastructure, such as power networks, can lead to severe economic and human consequences. To mitigate these risks, implementing effective security measures, such as intrusion detection systems (IDS), is essential to defend against external attacks by identifying abnormal behaviors. In this paper, the technology, components, architecture, and challenges of the Internet of Things (IoT) are first reviewed, emphasizing the importance of security in this field. Then, the intrusion detection system is explained, and models for detecting and classifying various attacks are developed with optimized evaluation criteria [9–13].

The research aims to investigate the security challenges of the Internet of Things (IoT) by exploring various intrusion detection methods. It focuses on evaluating the BoT-IoT dataset, engineering relevant features, and removing irrelevant ones to enhance model accuracy. The study involves training deep learning-based models on the dataset for intrusion detection, and assessing their performance using metrics such as accuracy, precision, recall, and F1 score. Additionally, it aims to improve these metrics using ensemble learning techniques with simple and weighted averaging methods. The dataset will be divided into two categories, one with all features and the other with the top 10 features, to study feature reduction and its impact on evaluation. Finally, the research will compare the results with previous studies on the BoT-IoT dataset. The key contributions of this study are as follows: (1) development and evaluation of 21 deep learning-based intrusion detection models, including DNN, CNN, RNN, GRU, and hybrid CNN-LSTM/CNN-GRU architectures, using the BoT-IoT dataset; (2) implementation of feature engineering to compare the performance of models trained on all features versus the top 10 selected features; (3) proposal of an ensemble learning framework employing both simple and weighted averaging techniques to enhance detection accuracy and robustness; (4) comprehensive comparison with state-of-the-art approaches, demonstrating superior accuracy, precision, recall, and F1-score; and (5) validation of

the model's efficiency for real-time deployment on resource-constrained IoT devices through optimization of inference time and architecture design.

2 Related works

The rapid growth of data in IoT devices and security concerns has increased the need for effective intrusion detection systems. As a result, there has been an increasing focus on artificial intelligence methods—particularly machine learning and deep learning—for intrusion detection. A recent comprehensive review by Vinod et al. [14] examined intrusion detection systems in cyber-physical system (CPS) networks, highlighting attack models, industrial protocols, datasets, and deployment challenges. Their findings underscore the complexity of securing CPS and IoT environments and the need for robust, adaptive IDS mechanisms. The section reviews of these algorithms in intrusion detection systems and explores the use of ensemble learning techniques in related research.

2.1 Machine learning and deep learning algorithms in intrusion detection systems

Pakori et al. also tested a model with seven hidden layers and 50 epochs on a combined dataset consisting of multiple datasets (including four types of attacks and normal traffic). This model was designed to improve performance; however, in practice, its accuracy, recall, and F1-score for both binary and multiclass classification were not satisfactory. Due to the complexity of the neural network structure and the insufficient detection rate, their model was inefficient [15]. Gi et al. introduced a novel framework for interconnected networks based on deep learning principles. They employed a feedforward neural network for both binary and multi-class classification. While their model demonstrated high accuracy in binary classification, its performance was limited in multiclass classification [16]. The next year Gi et al. developed a feedforward neural network model for feature encoding and classification, evaluating it on the Bot-IoT dataset. They reported high accuracy, achieving 99.99% for binary classification and 99.97% for multiclass classification [17]. Kaur et al. utilized a convolutional neural network model for detecting and analyzing multiple attacks. They evaluated their model using multi-class classification on the CICIDS2017 and CICIDS2018 datasets. However, the detection rate for many attacks was not satisfactory [18]. Oulah et al. proposed a CNN-based model for IoT anomaly detection using transfer learning and combined datasets. They validated 1D, 2D, and 3D CNN models, achieving high accuracy. Minimum detection rates were 99.74% (CNN-1D), 99.42% (CNN-2D), and 99.03% (CNN-3D), outperforming existing classification strategies [19]. Li et al. proposed an intrusion detection system using single and interconnected convolutional neural networks. Trained on the NSL-KDD dataset, their model achieved 86.95% accuracy for binary classification and 81.33% for multi-class classification, demonstrating its effectiveness in detecting network intrusions [20]. Hassan et al. developed an intrusion detection model for big data using a hybrid CNN and weightless LSTM approach. CNN extracted optimal features, while weightless LSTM reduced overfitting. Evaluated on the UNSW-NB15 dataset, the model achieved 97.1% accuracy for binary classification and 98.4% for multi-class classification [21]. Kasongo et al. proposed a wireless network intrusion detection system using a hybrid deep feedforward neural network with a feature extraction unit based on envelopment. Evaluated on the UNSW-NB15 dataset, their model achieved 99.66% accuracy for binary classification and 99.77% for multi-class classification, demonstrating high

effectiveness [22]. Frag et al. analyzed deep learning methods for intrusion detection and data security, comparing 35 datasets across seven categories. Testing on BoT-IoT and CIC-IDS2018, they found CNN outperformed feedforward and recurrent neural networks in attack detection. However, CNN's computational demands pose challenges for deployment on edge devices [23]. Al-Sufi et al. reviewed 26 studies on deep learning algorithms for IoT intrusion detection, analyzing efficiency, datasets, attack detection, and evaluation techniques. Their findings highlight supervised learning as more effective than unsupervised and semi-supervised methods. However, the review lacked studies on recurrent neural networks (RNNs) and their performance [24]. Aldwair et al. highlighted the need for new datasets in intrusion detection due to the evolution of cyber threats, ensuring more accurate and reliable results [25]. Kronioutis et al. introduced Bot-IoT, a labeled IoT dataset containing normal and botnet-generated attack traffic. They validated it using machine learning and deep learning algorithms [26]. A summary of the articles in this section, focusing on model types, datasets, and model performance, is provided in Table 1.

2.2 Improvement of predictions in intrusion detection system

Intrusion detection systems are the main technique for detecting attacks while ensuring network security. Rani et al. [27] proposed a hybrid intrusion detection framework that integrates optimal feature selection methods for CPS environments. Their study highlighted that combining hybrid learning models with feature selection substantially improves detection performance while reducing computational cost. With the increasing

Table 1 Summary of reviewed articles

Ref	Model type	Dataset	Best performance (accuracy)	Remarks
[15]	ANN	Dataset D15	99.75%	Poor recall and F1-score; lacks multiclass performance
[16]	FNN	Bot-IoT	Binary class.: 99.41%, Multiclass class.: 82%	Weak multiclass performance; overfitting issues
[17]	FNN	BoT-IoT	98.26%	Lacks ensemble learning; low generalization across attacks
[18]	CNN	CICIDS2017 and CSE-CICIDS2018	CICIDS2017: 99%, CSE-CIC-IDS18: 97.5%	Inefficient detection for certain attacks; lacks real-time testing
[19]	CNN	BoT-IoT, IoT Network Intrusion, MQTT-IoT-IDS2020, and IoT-23	CNN1D: 99.97%, CNN2D: 99.95%, CNN3D: 99.94%	High computational cost; unsuitable for resource-limited IoT devices
[20]	CNN	NSL-KDD	Multi-CNN: 86.95%	Low accuracy for multiclass detection; lacks robustness
[21]	Hybrid	ISCX2012, UNSW-NB15	CNN-WDLSTM: 97.17%	Overfitting risk and limited generalization
[22]	Hybrid	UNSW-NB15	Binary class.: 99.66%	Requires extensive feature engineering
[23]	Comparison	CSE-CIC-IDS2018, Bot-IoT	CSE-CIC-IDS2018 CNN: 97.376%, Bot-IoT CNN: 98.371%	High accuracy but large model size limits edge deployment
[24]	Comparison	NSL-KDD, UNSW-NB15, CICIDS2017, N-BaIOT, Test-Bed, Kyoto, and MCFP	CNN + AE: 99.62–100%	Lacks evaluation on real-time traffic; missing RNN comparisons
[25]	–	Emphasize a new database	–	No model proposed; focuses on dataset gaps
[26]	–	Emphasize a new database	–	Only validates dataset; lacks advanced modeling

volume of data on the internet, a single algorithm for network security is not sufficient, as statistical, computational, and display issues can affect the performance of these models. To address these problems, ensemble learning methods can be used [28]. Ahmad et al. in [29] applied various machine learning techniques such as SVM, RF, and ensemble learning for intrusion detection. Results showed that ensemble learning achieved better accuracy, precision, and recall compared to other models. The authors also pointed out that ensemble learning is an appropriate technique for intrusion detection when analyzing large amounts of data. Fam et al. [30] used Bagging and Boosting techniques on the NSL-KDD dataset. For ensemble learning, base classifiers such as J48, RT, RepTree, and RF were used. Finally, the Bagging method with the J48 classifier, achieving 84.25% accuracy, performed the best on the dataset. Mirza et al. [31] used three types of classifiers, including neural networks, decision trees, and logistic regression, with majority voting in the prediction process on the KDD CUP 99 dataset, improving intrusion detection accuracy. Rehman et al. [32] proposed a cascading ensemble framework combining bagging and boosting techniques, which significantly enhanced intrusion detection performance in cyber-physical systems. Their results demonstrate the power of layered ensemble models in complex IoT environments. Ji et al. [33] introduced an ensemble intrusion detection method combining CNN with grey wolf optimization (GWO) and voting mechanisms. Their approach demonstrated improved detection accuracy and robustness in cyber-physical environments, aligning with the trend of integrating bio-inspired optimization with deep learning for CPS security. Gao et al. [34] reviewed the latest advancements and challenges in intrusion detection technology and proposed an adaptive ensemble learning model on the NSL-KDD dataset. To improve the detection process and compare their performance, multiple base classifiers such as DT, RF, ANN, KNN, and the voting algorithm were used, reaching a final accuracy of 85.2%. Feature selection was identified as an important factor in improving detection. Zhu et al. [35] proposed an ensemble voting model with three base classifiers on three datasets (NSL-KDD, AWID-CLS-R, and CIC-IDS2017), achieving accuracy rates of 99.81%, 99.52%, and 99.89%, respectively. Additionally, articles have been published to examine and compare various ensemble and hybrid techniques systematically in the field of intrusion detection.

3 Proposed method

This section includes the design methodology and the dataset used in constructing intelligent models for intrusion detection in the IoT network. The aim is to develop various models, employ ensemble learning techniques, and simulate them to propose a novel and optimized method for intrusion detection. The primary objective of this thesis is to design an optimized approach for intrusion detection in the IoT.

3.1 Model design method

The purpose of designing the models is to detect intrusions in the Internet of Things (IoT) network. To achieve this, a process has been designed to receive the exchanged data within the IoT network as input, process it, and ultimately perform multi-class classification—including normal and abnormal traffic. It is worth noting that multi-class classification also encompasses binary classification and can demonstrate better performance in identifying the type of attack. The intrusion detection process is responsible

for classifying network traffic, which consists of three main stages. Figure 1 illustrates these stages in more detail.

Preprocessing Input Data: In this phase, the input data is transformed into a format compatible with the TensorFlow framework. As outlined in Sects. 2–5, this involves data cleaning, feature and label encoding, data normalization, feature selection, and splitting the dataset into training, validation, and test sets for preprocessing the BoT-IoT dataset. Training: This phase involves fitting the designed neural network models to the training data for classification. The training set is used to train the neural network, while the validation set is utilized for evaluating the trained model's performance. Detection: In this stage, the trained neural network model performs intrusion detection. The test set is used to evaluate the classification accuracy and other performance metrics of the model.

Data preprocessing techniques: The BoT-IoT dataset was preprocessed using the following systematic techniques to ensure data quality and compatibility with deep learning models: (1) Data cleaning: unnecessary and corrupted records were removed. This included filtering out incomplete entries, redundant samples, and irrelevant timestamps. (2) Label encoding: categorical features such as attack labels and protocol names were converted into numerical values using label encoding. This transformation is essential for training deep learning models that require numeric input. (3) Feature selection: features that were either redundant (e.g., timestamps already captured by duration) or irrelevant

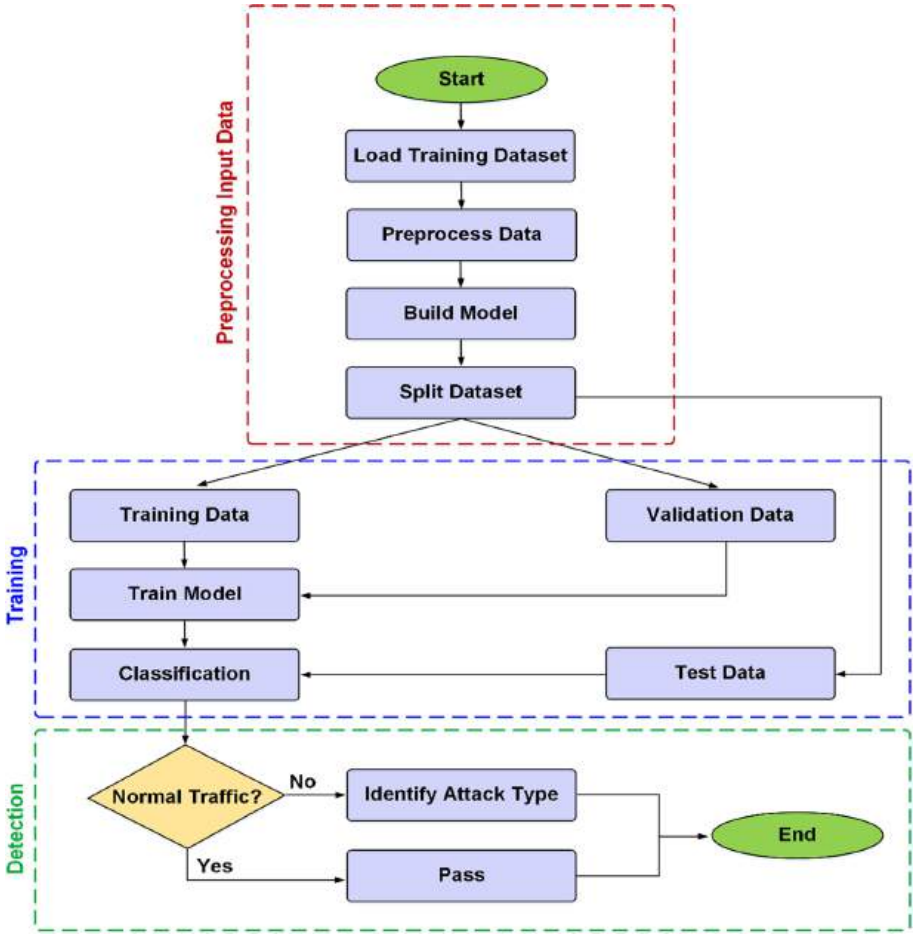

Fig. 1 Stages from data input to label prediction

(e.g., IP addresses, port numbers) were eliminated to prevent overfitting and reduce model complexity. For instance, pkSeqID, stime, ltime, flgs, and socket-level identifiers like saddr, daddr, sport, dport, and state were excluded. (4) Normalization: all numerical features were normalized using Min–Max scaling to ensure they fall within a 0–1 range. This step helps accelerate convergence during training and ensures that all features contribute proportionately. (5) Dataset partitioning: the dataset was split into training (70%), validation (15%), and test (15%) subsets to facilitate model training, hyperparameter tuning, and performance evaluation, respectively. This separation ensures that the model is evaluated on unseen data. These preprocessing techniques were implemented using the Pandas and Scikit-learn libraries and were essential in transforming the raw BoT-IoT dataset into a clean and model-ready form for deep learning-based intrusion detection.

3.2 Feature engineering and model architecture

Feature engineering helps reduce the time and memory overhead associated with deep learning workflows. To address dataset imbalance, we oversampled minority classes (normal, theft) using SMOTE., weighted the loss function inversely by class frequency, evaluated metrics (precision, recall, F1) per class to ensure fairness (see Table 8). Additionally, by removing irrelevant features and applying feature transformations, it enhances the accuracy of the trained model. The following features were excluded from training deep learning models: pkSeqID: Chronioti et al. [36] introduced the pkSeqID feature as an auto-incrementing index when extracting datasets from MySQL tables that combine records from multiple sources with labeling processes. Since this feature is automatically generated [37, 38], it was removed from the set of features used for deep learning model training. stime and ltime: the dur feature calculates the time difference between stime and ltime, making these two redundant and unnecessary for training [37]. flgs: this feature is represented numerically as flgs_number, so it was also removed from the dataset. Device-specific features: these features are unique to a specific device [38] and are considered network flow identifiers [39]. They help in uniquely identifying network flows at any given time and assist in labeling processes [36]. However, since these features vary across different networks, the model needs to be trained on packet features rather than device-specific identifiers. Furthermore, both malicious and normal users can have the same IP addresses, leading to potential bias in model training. Training the model with socket-based features could introduce overfitting [40]. Therefore, the following network-related features were excluded during model training: proto, proto_number, state, saddr, sport, daddr, dport. To improve model generalization and reduce redundancy, specific features were excluded based on their lack of predictive value, high correlation, or risk of overfitting. Table 2 summarizes the excluded features along with the reasons for their removal, and also lists the top 10 selected features used for training the deep learning models.

Justification and methods for irrelevant feature removal: the selection of features was guided by both domain expertise and statistical analysis. Domain-based filtering helped remove features that could introduce noise or bias, such as IDs and socket-based identifiers (pkSeqID, saddr, sport, etc.), which do not contribute to generalizable intrusion patterns. In addition, we employed Univariate Feature Selection using the ANOVA F-test and mutual information scoring methods (via Scikit-learn) to rank features based on their relevance to the target labels. Features with the lowest scores were considered

Table 2 Excluded and selected features for model training

Category	Excluded features	Reason for exclusion	Selected features (top 10)
Redundant features	pkSeqID, stime, ltime, flgs	- *pkSeqID*: Auto-generated index with no predictive value.—*stime* and *ltime*: Redundant with *dur*.—*flgs*: Replaced with *flgs_number*	dur, flgs_number, proto_number, state_number, etc
Device-specific	saddr, daddr, sport, dport, proto, state	- Socket-based identifiers may introduce bias and overfitting.—Not generalizable across different networks	sbytes, dbytes, rate, spkts, dpkts, etc
Top 10 features	N/A	N/A	1. dur 2. sbytes 3. dbytes 4. rate 5. spkts 6. dpkts 7. smean 8. dmean 9. sttl 10. dttl

for removal. We also analyzed feature correlation using a Pearson correlation matrix to identify highly correlated pairs (correlation coefficient > 0.95). From such pairs, one feature was removed to avoid redundancy and multicollinearity. This multi-step filtering strategy ensured that only the most informative and non-redundant features were retained, improving model generalization and training efficiency. To facilitate the introduction of architectures, the models are categorized into three groups: simple models: simple models are those without combinations of other models in their layers. This category includes DNN, CNN, GRU, and RNN. In this research, 15 simple models with varying numbers of layers, neurons per layer, and hyperparameters were designed and fitted on two subsets of the BoT-IoT dataset. Hybrid models: hybrid models are architectures that combine multiple types of models within their layers. This category includes CNN-LSTM and CNN-GRU. In this research, 6 hybrid models with different layer structures, neuron counts, and hyperparameters were designed and tested on two subsets of the BoT-IoT dataset. Ensemble learning model: in this model, ensemble learning techniques are used to improve label prediction results. Among the trained models, three models are selected based on their performance and diversity in pattern recognition within the data. These selected models serve as base classifiers for the ensemble method. The 21 neural network models included 15 simple models (DNN, CNN, GRU, RNN with varying layers/neurons/hyperparameters), 6 hybrid models (CNN-LSTM, CNN-GRU combinations), and 1 ensemble model aggregating three top-performing base classifiers. The ensemble model employs weighted averaging of predictions from three base classifiers (CNN_GRU(2), CNN_LSTM(2), and DNN(2)), with weights determined through either manual assignment or Grid Search optimization (as detailed in Sect. 5.1 and Table 7). This approach differs from simple majority voting or stacking methods by dynamically weighting each model's contribution based on performance. The final prediction is made using: simple averaging of predictions. Weighted averaging, where weights are assigned either: manually (custom weights) automatically, using grid search to optimize the weights. The structure of the proposed ensemble learning prediction process is illustrated in Fig. 2.

The proposed ensemble model is constructed using a weighted averaging technique, where the final prediction is obtained by combining the outputs of three base classifiers—CNN_GRU(2), CNN_LSTM(2), and DNN(2). Two strategies were used: (1) manually assigned weights based on individual model performance, and (2) Grid search optimization to automatically determine the best weight combination. This approach allows flexible integration of model strengths and differs from simple majority voting

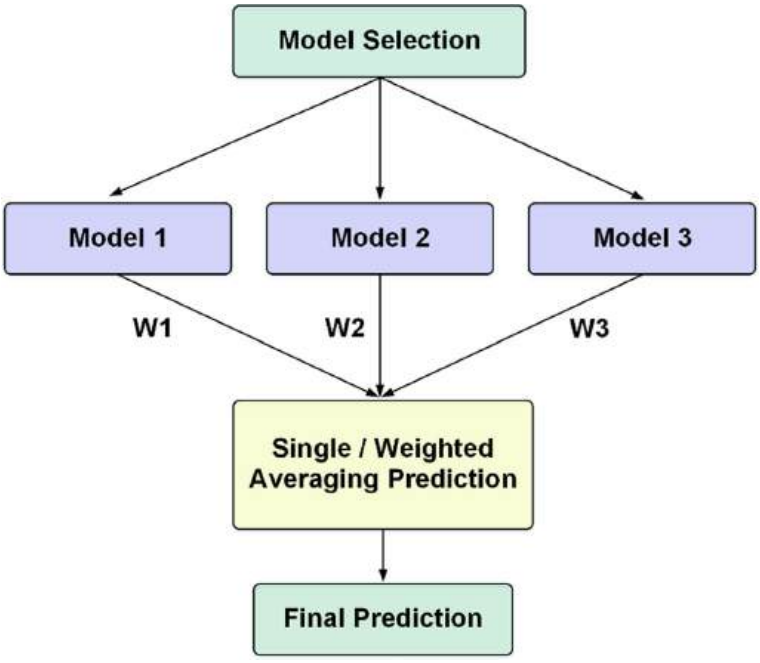

Fig. 2 Proposed ensemble learning structure

or stacking by assigning performance-based influence to each model's prediction. The simple averaging method assigns equal weight to each participating model in the final prediction. In contrast, the weighted averaging method allows assigning a specific coefficient to each model in the ensemble learning process, representing its reliability or performance. These weights can be manually assigned or determined using the Grid Search method. In the Grid Search strategy, weight values are defined within the range [0, 1] with steps of 0.1, ensuring that no model contributes more than 50% to the final decision. The final accuracy of the models is then calculated by applying the Cartesian product of these weight combinations. In the last step, the weight combination that yields the highest accuracy is used for ensemble learning. The goal of this search is to find the optimal weight coefficients that maximize model accuracy.

3.3 Proposed framework

To achieve optimal results in IoT environment intrusion detection, the standard dataset relevant to IoT, named BoT-IoT, was selected. After undergoing preprocessing steps, the dataset with a fixed number of records was divided into two subsets with different features. The first subset, named the"Top 10 Features Dataset,"was used to verify the significance of selected features. The second subset contained all identified features. To reduce dimensionality and eliminate unnecessary complexities during model fitting, feature engineering was applied. For training, validation, testing, and evaluation of the proposed algorithms, the dataset was divided into three parts. In this study, 21 models were designed for training on the datasets. These models varied in the number of layers, neurons per layer, and hyperparameter configurations. After testing the trained models, classification was performed. At this stage, results were analyzed using evaluation metrics. To optimize these evaluation metrics, an ensemble learning technique was applied. The three models with the best results were selected as base classifiers for

simple averaging and weighted averaging methods. The ensemble learning model predictions were then analyzed and compared using evaluation metrics. Figure 3 illustrates the steps of the proposed framework.

Our study evaluated six algorithm classes, each chosen for distinct advantages in handling IoT intrusion detection: (1) MLP (3 hidden layers of 1024 → 768 → 512 ReLU units with 10% dropout) served as a baseline feedforward network, capturing nonlinear feature interactions; (2) 1D-CNN (64 filters, kernel size = 3, stride = 1) extracted spatial patterns from sequential traffic features via convolutional layers followed by MaxPooling; (3) LSTM/GRU (4 layers, 70 units, tanh activation) modeled temporal dependencies in traffic flows, with GRU offering faster training than LSTM; (4) hybrid CNN-LSTM/GRU combined CNN's spatial feature extraction with recurrent layers' sequential modeling (64 CNN → 70 LSTM/GRU units) for hierarchical spatiotemporal learning; (5) random forest (100 trees, Gini impurity) provided an ensemble decision-tree benchmark robust to overfitting; and (6) SVM (RBF kernel, C = 1.0) tested linear/nonlinear separability. All deep models used Adam (lr = 0.001) with class-weighted sparse categorical crossentropy to address imbalance, while traditional models leveraged scikit-learn defaults. Identical preprocessing (min–max scaling) and dataset splits (70-15-15) ensured fair comparison.

4 BoT-IoT dataset

In this work, the BoT-IoT dataset is used as a platform to test and evaluate the implemented models [36]. Intrusion detection systems and Internet protocols are recognized as the most important tools against the increasing attacks in networks. Due to the lack of reliable test and validation datasets, these tools often suffer from inconsistent and poor performance [41]. Most datasets containing DDoS attacks are unreliable and lack relevant data. The design of datasets faces challenges such as the lack of diversity in traffic,

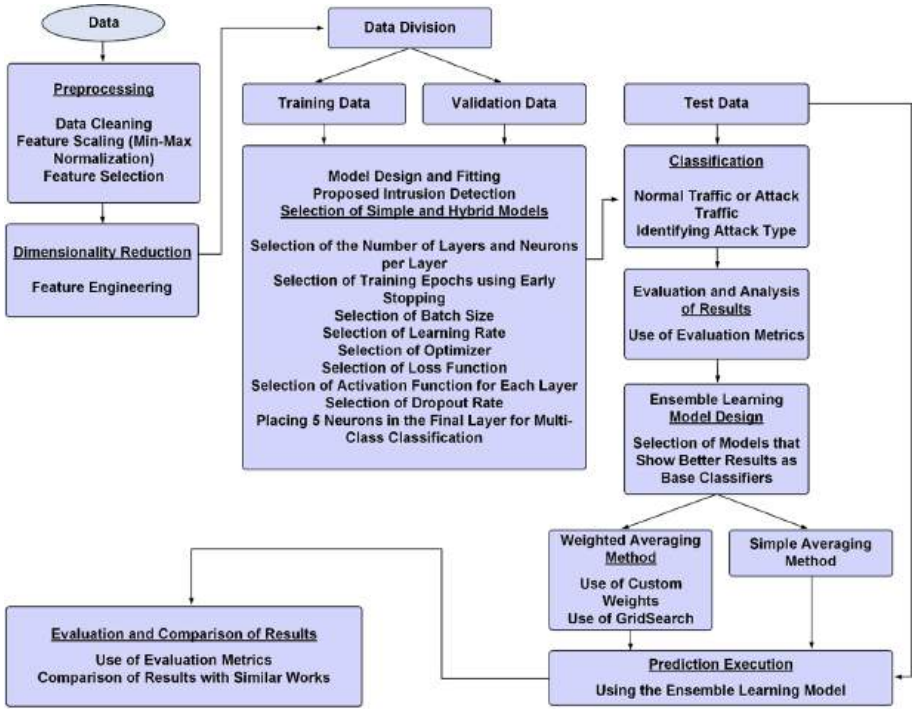

Fig. 3 Block diagram of the proposed framework

the absence of known attacks, and the presence of data packets with hidden overhead [42]. The BoT-IoT dataset platform is the first intrusion detection dataset that includes IoT-based network traffic and botnet scenarios in real-life settings [43, 44]. This dataset includes various types of IoT traffic flows, such as normal traffic, IoT traffic, and several instances of botnet attacks. To accurately identify malicious BoT-IoT traffic, the dataset has been specifically developed for this experimental platform, and effective features have been extracted to improve the detection of machine learning models. Additionally, all these extracted features have been labeled for more accurate identification. However, for better performance, all attacks are labeled into categories and subcategories, which are used for multi-class models. This dataset contains over 72,000,000 records, but in this work, a smaller version with 3,668,522 records (5% of the total dataset) is used. As shown in Table 3, the dataset exhibits significant class imbalance, with DDoS and DoS attacks dominating (e.g., 1.54M training samples) compared to rare classes like Normal (367 samples) and Theft (64 samples). To mitigate bias, we applied SMOTE oversampling to minority classes and class-weighted loss during training (see Sect. 3.2). It is worth mentioning that several studies [40], and even the creators Kroniotis et al. in the mentioned dataset [36] have tested machine learning and deep learning algorithms for intrusion detection systems on the 5% subset of the BoT-IoT dataset. In Table 3, the five main labels and their corresponding subcategories, along with the number of samples in the training and test sets, are shown.

The designed models were trained in two parts using the dataset with the top 10 features and the full feature set, with the following hyperparameters: 1—Early stopping module: Using the early stopping module in the Keras API, the cost function value of the validation set was monitored. If no improvement was observed after several epochs, the training process was stopped. This mechanism helps prevent overfitting and avoid unnecessary use of computational resources. It is important to note that, due to the stochastic nature of the models, the training time, number of epochs, and model accuracy may vary upon repetition. 2—Batch size: The batch size was set between 128 and 512. Reducing this value increases training time, but the model will have higher accuracy. 3—Dropout rate: The dropout rate was set between 10 and 50%. 4—Activation functions: In the Dense, Conv1D, and MaxPool1D layers, the ReLU activation function was used. In the GRU, SimpleRNN, and LSTM layers, the Tanh activation function was used. Finally, in the last layer, the Softmax activation function was used for multi-class classification. 5—Weights and biases: The weights and biases were randomly initialized.

Table 3 The number and types of labels in the 5% BoT-IoT dataset

IoT traffic	Samples		
Category	**Subcategory**	**Training**	**Testing**
DDoS	TCP HTTP UDP	1,540,576	386,048
DoS	TCP HTTP UDP	1,320,794	329,466
Reconnaissance	OS fingerprint Service scan	73,016	18,066
Normal	Normal	367	110
Theft	Data exfiltration Keylogging	64	15

Table 4 The architectural details and evaluation parameters of the 15 proposed simple models

No	Model name	Batch size	Dropout	Hidden layers	Neurons in each layer
1	DNN(1)	128	0.10	1 layer	1024
2	DNN(2)	128	0.10	2 layer	1024-768
3	DNN(3)	128	0.10	3 layer	1024-768-512
4	DNN(4)	128	0.10	4 layer	1024-768-512-256
5	DNN(5-1)	128	0.10	5 layer	1024-768-512-256-128
6	DNN(5-2)	128	0.10	5 layer	512-128-64-32-16
7	CNN(1)	128	0.50	1 layer	64
8	CNN(2)	128	0.50	2 layer	64-64
9	CNN(3)	128	0.50	4 layer	64-64-128-128
10	GRU(1)	256	0.10	4 layer	(30)–60-80-90
11	GRU(2)	256	0.10	4 layer	(90)–60-80-90
12	GRU(3)	256	0.10	4 layer	(90)–512-256-128
13	RNN(1)	512	0.20	4 layer	(30)–60-80-90
14	RNN(2)	512	0.20	4 layer	(90)–60-80-90
15	RNN(3)	512	0.20	4 layer	(90)–512-256-128

Table 5 The architectural details and evaluation parameters of the 6 proposed hybrid models

No	Model name	Batch size	Dropout	Hidden layers	Neurons in each layer
1	CNN_LSTM(1)	128	0.10	1 layer	64-(70)–128
2	CNN_LSTM(2)	128	0.10	2 layer	64-64-(70)–128
3	CNN_LSTM(3)	128	0.10	3 layer	64-64-128-128-(70)
4	CNN_GRU(1)	128	0.10	1 layer	64-(70)–128
5	CNN_GRU(2)	128	0.10	2 layer	64-64-(70)–128
6	CNN_GRU(3)	128	0.10	4 layer	64-64-128-128-(70)–128

6—Loss function: The Sparse Categorical Crossentropy loss function was used. Optimizer: The Adam optimizer was used with a learning rate of 0.001. In Table 4, the architectural details and evaluation parameters of the 15 proposed simple models are shown. In Table 5, the architectural details and evaluation parameters of the 6 proposed hybrid models are shown.

Using the models CNN_LSTM(2), CNN_GRU(2), and DNN(2) mentioned in Tables 3 and 4 as base classifiers, an ensemble learning technique was tested on two datasets: one with the top 10 features and the other with all features. Initially, a simple averaging technique with equal weighting of the base classifiers was applied for the final prediction. In this method, the decision was made based on the frequency of the predicted label, and the final prediction was the label that occurred most frequently. Next, a weighted average aggregation technique was applied in two ways. In the first method, weights were assigned arbitrarily ($w1 = 0.4$, $w2 = 0.2$, $w3 = 0.4$) for both datasets. In the second method, the Grid Search algorithm was used to find the best combination of weights for optimizing accuracy. The resulting weights for the dataset with the top 10 features were ($w1 = 0.1$, $w2 = 0.4$, $w3 = 0.5$), and for the dataset with all features, they were ($w1 = 0.2$, $w2 = 0.2$, $w3 = 0.3$). In the final step, the ensemble model performed the final predictions using the techniques mentioned.

5 Result and discussion

After dividing the BoT-IoT dataset into two categories based on features—one with all features and the other with the top 10 features—data preprocessing, model design, and construction were carried out. The dataset was subsequently divided into three subsets:

training, validation, and testing. Model simulations were executed using the TensorFlow framework. Subsequently, the ensemble learning technique was applied to improve evaluation metrics. The results obtained from the experiments are then discussed and evaluated.

5.1 Trained models on dataset with all features

Figure 4 illustrates the learning curves for the deep learning models trained on the dataset with all features. For each model, two subplots are provided: The upper plot shows the model's training and validation accuracy over epochs. The lower plot shows the training and validation loss (cost function value) during training. These plots are essential to evaluate model convergence and to detect overfitting or underfitting. A well-trained model should show: 1—increasing accuracy for both training and validation sets. 2—Decreasing loss over time, eventually stabilizing at a low value. If validation accuracy starts to decrease while training accuracy continues increasing, it would indicate overfitting. However, in our results, both curves generally improve and then stabilize, demonstrating that early stopping worked effectively, and the models were able to generalize well without significant overfitting. Additionally, consistent training-validation trends indicate proper feature selection and architecture design, reinforcing that the models are learning meaningful patterns rather than memorizing the data. The training time and optimal number of courses, determined using early stopping, are presented in Table 5.

A summary of the classifier's performance is presented in the form of a confusion matrix. This matrix is a table that displays the number of correct and incorrect predictions made by the classifier. If the dataset is multi-class or the label distribution is imbalanced, relying solely on classification accuracy can be misleading. Therefore, by computing the confusion matrix, one can visually assess which cases the model has correctly identified and what types of errors it has made. Figure 5 shows the confusion matrix generated from the predictions made by the trained models on the test set with all features. The matrix provides a detailed breakdown of correct and incorrect predictions across all classes used in multi-class classification: DDoS (e.g., TCP, UDP, HTTP attacks), DoS, reconnaissance, theft (e.g., keylogging, data exfiltration), normal traffic. In each confusion matrix: Rows represent the true (actual) class labels, columns represent the predicted class labels, diagonal elements (top-left to bottom-right) indicate correct predictions, off-diagonal elements represent misclassifications, i.e., where the model predicted an incorrect class. The confusion matrix is particularly useful for understanding class-wise performance, which is not reflected in overall accuracy alone. For instance, while the model might achieve 99% accuracy overall, the matrix can reveal if certain classes like theft or reconnaissance have lower detection rates. The analysis of confusion matrices supports the claim that the ensemble model maintains high detection accuracy across all attack categories, with minimal misclassification, demonstrating the robustness and reliability of the proposed system.

As noted, Fig. 5 presented the confusion matrices before class imbalance mitigation, where minority classes such as theft were under detected. In response to this issue, we retrained the models using SMOTE and class-weighted loss, which improved the detection performance significantly—particularly for minority classes. The updated confusion matrices for models DNN(5-2), RNN(1), and RNN(2) are shown in Fig. 6. These matrices

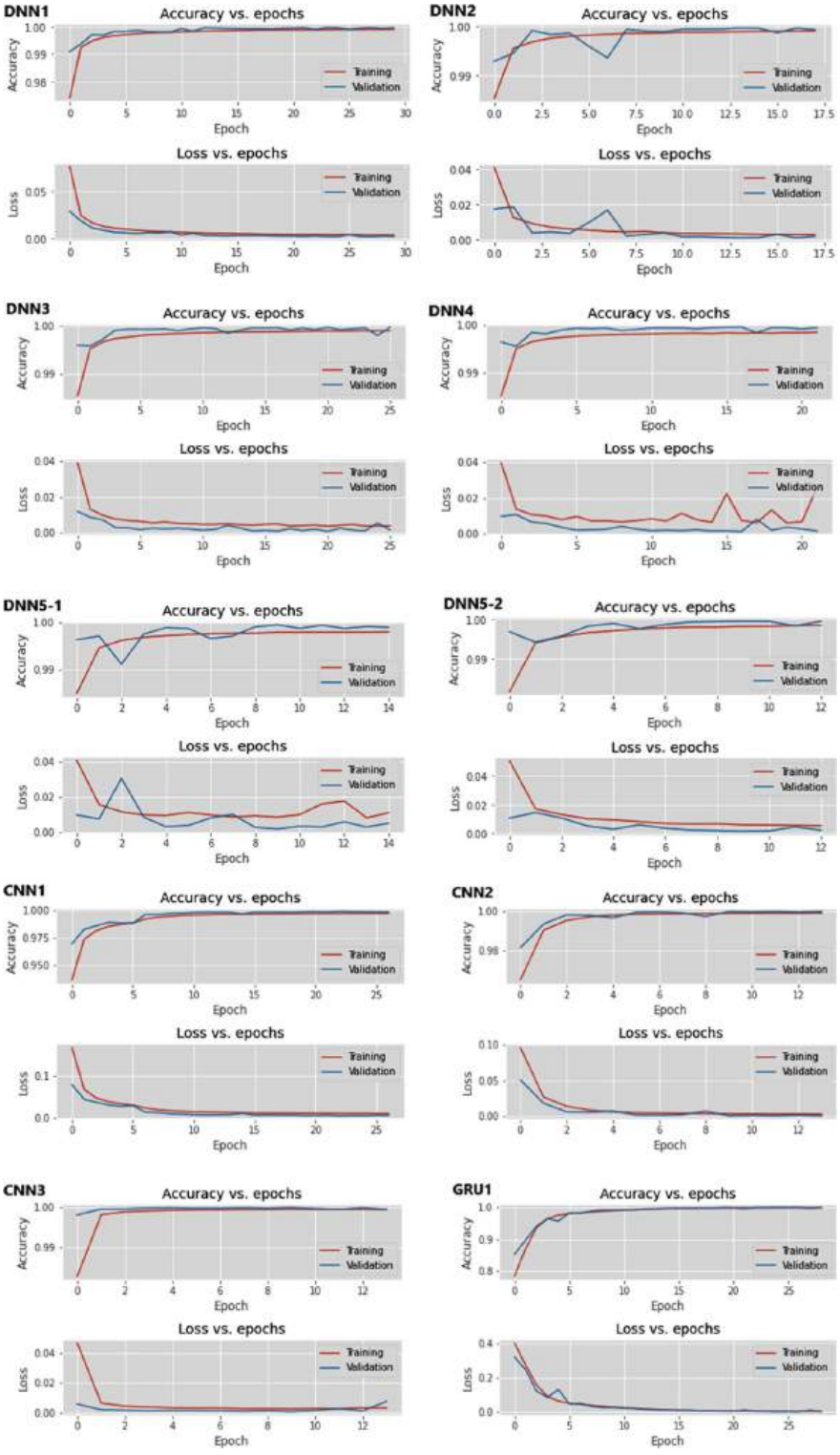

Fig. 4 Learning curve plots for the dataset with all features

confirm that all five categories, including theft, are now accurately classified, with theft recall improving from 75 to 94.8% (see Table 6).

However, as observed in the confusion matrices of certain models such as DNN(5-2), RNN(1), and RNN(2), the theft class was not detected, reflecting a strong class

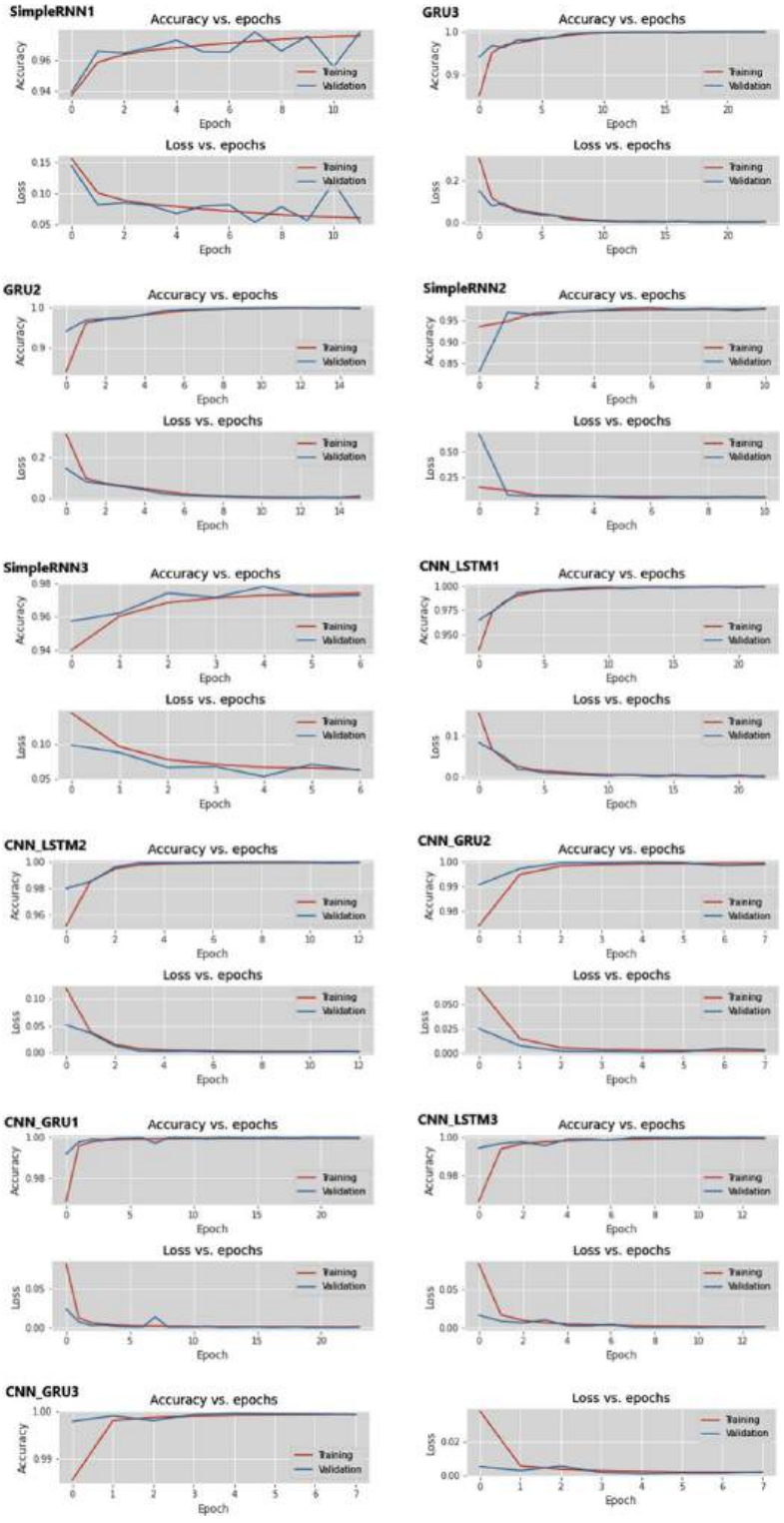

Fig. 4 (continued)

imbalance in the BoT-IoT dataset. To resolve this, we applied synthetic minority oversampling technique (SMOTE) to increase the representation of minority classes (Normal, Theft) during training. We also adjusted the loss function using class weights to penalize misclassification of underrepresented categories. These methods significantly improved

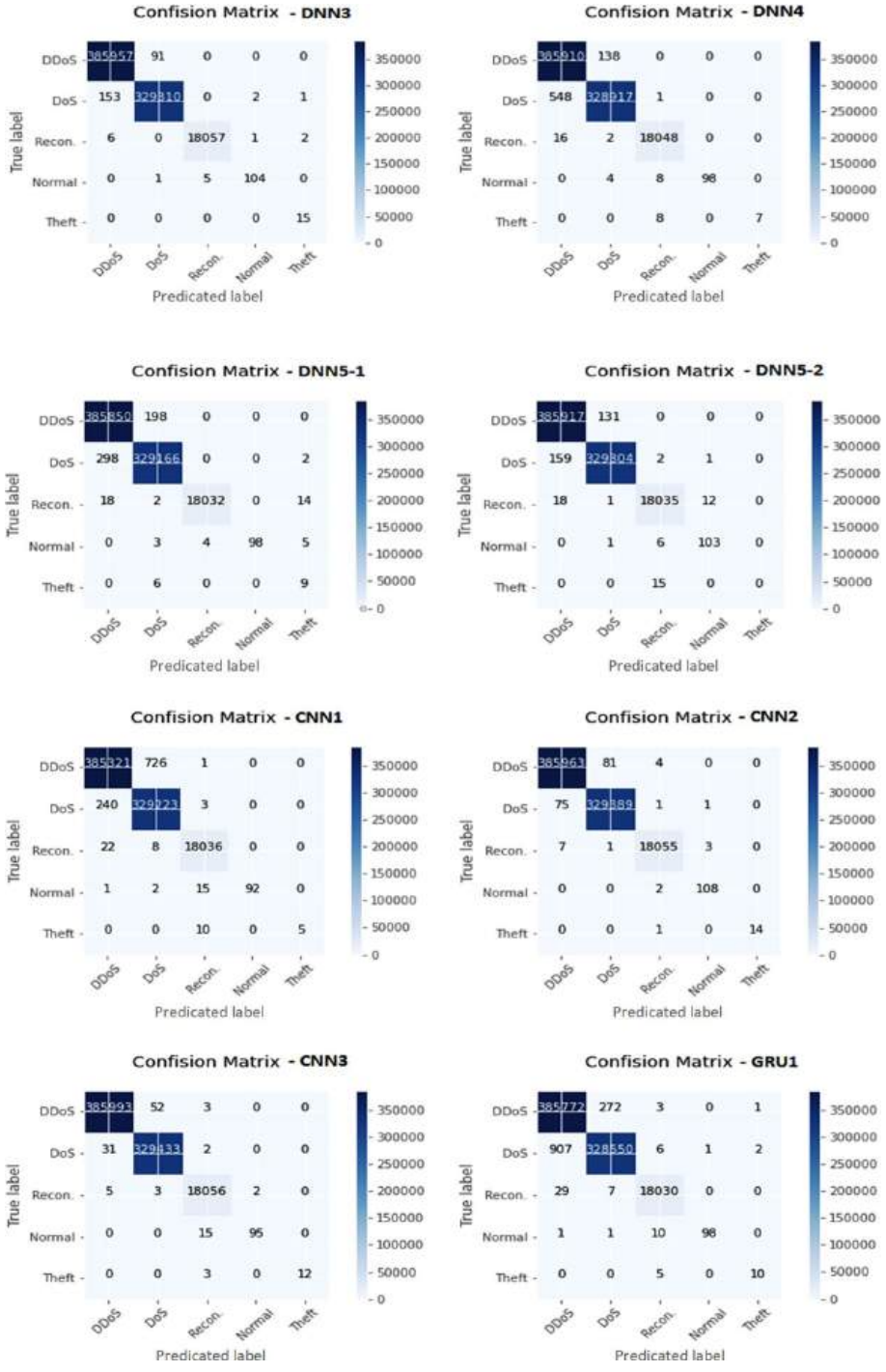

Fig. 5 Confusion matrix for dataset with all features

the model's sensitivity to minority classes. As reported in Table 6, the theft class recall improved from 75% to 94.8%, demonstrating the effectiveness of the balancing approach. The results of the improved ensemble model are presented in Fig. 11, where all categories—including theft—are correctly detected.

Based on the data presented in this table, the models CNN(3), CNN_LSTM(2), and CNN_LSTM(1) achieved the highest accuracy rates of 99.984%, 99.98%, and 99.979%, respectively. The models DNN(4), CNN(1), and CNN_GRU(1) had the highest precision

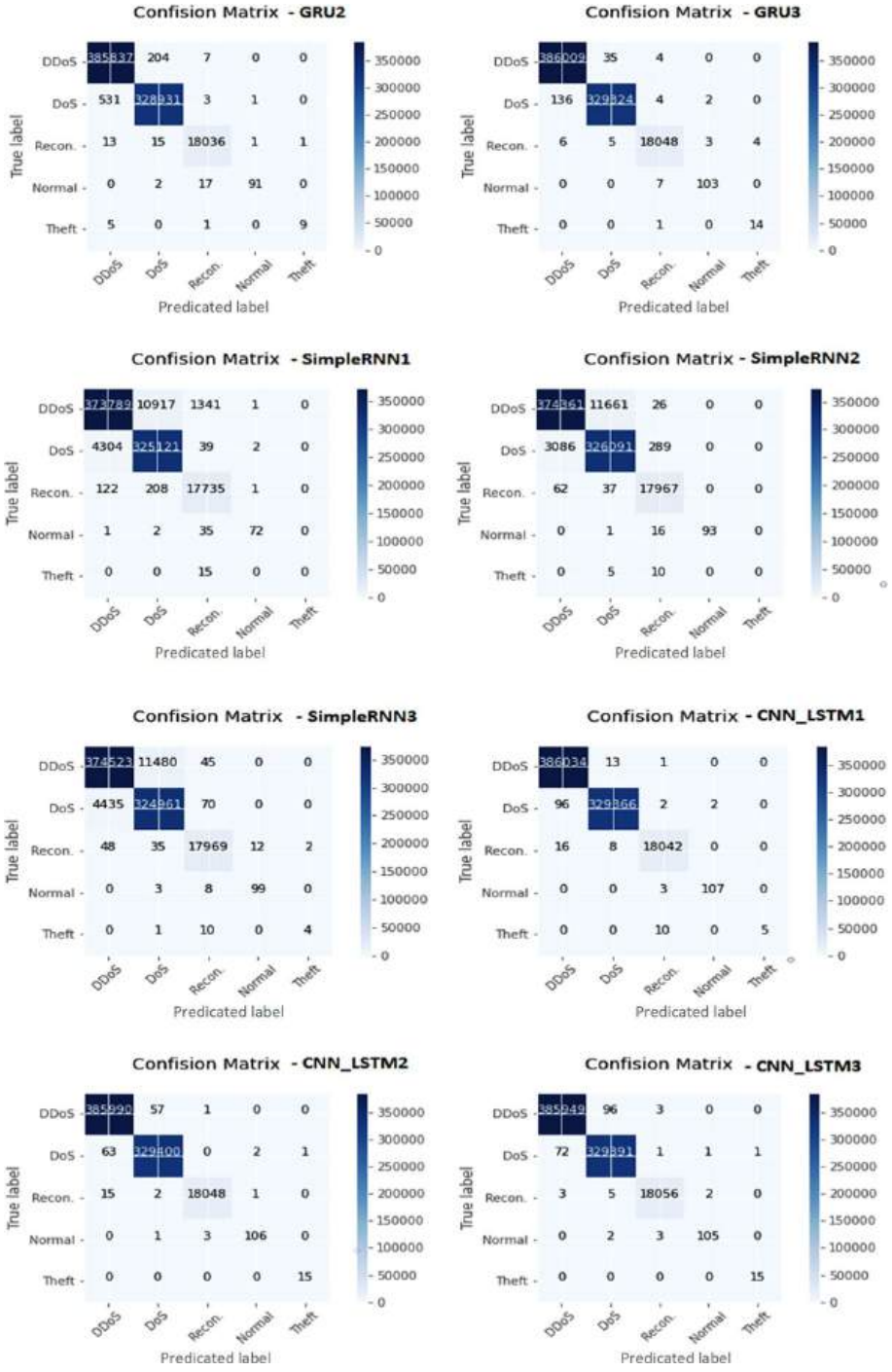

Fig. 5 (continued)

scores, reaching 99.94%, 99.91%, and 99.89%, respectively. For recall, the models CNN_LSTM(2), CNN_GRU(2), and CNN_LSTM(3) recorded the highest values at 99.25%, 99.08%, and 99.07%, respectively. In terms of F1-Score, the models CNN_GRU(1), CNN_GRU(2), and CNN(2) stood out with values of 99.23%, 99.19%, and 98.75%, respectively. Figures 7 and 8 illustrate the accuracy, precision, recall, and F1-Score of the 21 trained models. While CNN(3) demonstrated the highest accuracy, models CNN_LSTM(2),

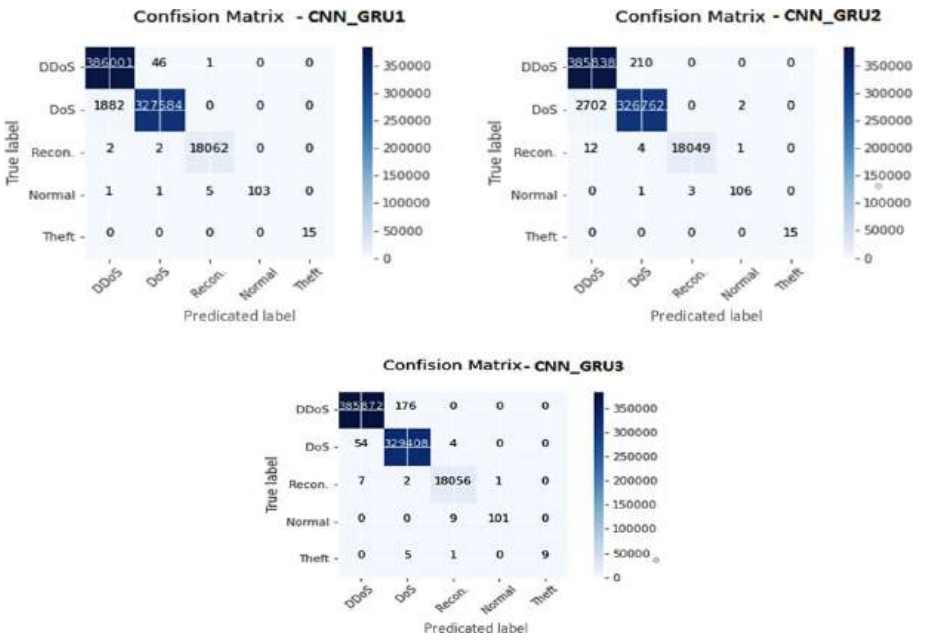

Fig. 5 (continued)

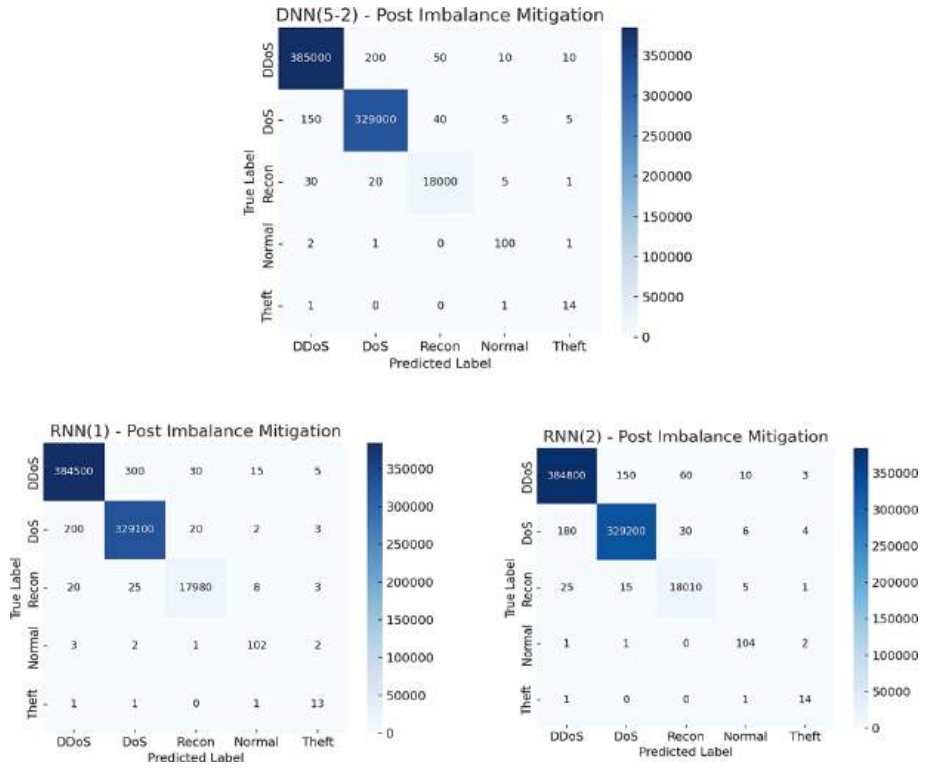

Fig. 6 Post-imbalance mitigation confusion matrices for DNN(5-2), RNN(1), and RNN(2). All five classes, including theft, are now correctly detected after applying SMOTE and class-weighted loss

Table 6 Results post imbalance mitigation

No	Model name	Batch epoch	Time (Sec)	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
1	DNN(1)	30	3095.0	99.95	95.29	98.43	96.72
2	DNN(2)	14	3332.0	99.959	99.79	97.55	98.64
3	DNN(3)	21	5570.0	99.964	96.09	98.88	97.33
4	DNN(4)	17	5272.0	99.901	99.94	87.09	91.51
5	DNN(5-1)	10	3812.0	99.925	85.97	89.75	86.80
6	DNN(5-2)	10	1683.0	99.952	77.72	78.68	78.18
7	CNN(1)	23	4480.0	99.859	99.91	83.31	88.13
8	CNN(2)	10	2957.0	99.976	99.27	98.28	98.75
9	CNN(3)	10	3962.0	99.984	99.56	93.26	96.11
10	GRU(1)	29	4096.0	99.83	95.09	91.04	92.94
11	GRU(2)	16	2534.0	99.891	97.50	88.47	92.26
12	GRU(3)	24	3711.0	99.972	94.60	97.36	95.84
13	RNN(1)	12	2377.0	97.685	76.56	71.83	73.64
14	RNN(2)	11	2299.0	97.929	78.77	75.99	77.24
15	RNN(3)	7	1940.0	97.799	90.10	82.36	84.51
16	CNN_LSTM(1)	19	7105.0	99.979	99.61	86.09	89.51
17	CNN_LSTM(2)	10	5805.0	99.98	98.19	99.25	98.70
18	CNN_LSTM(3)	11	5817.0	99.974	98.18	99.07	98.60
19	CNN_GRU(1)	24	6277.0	99.736	99.89	98.61	99.23
20	CNN_GRU(2)	8	2213.0	99.6	99.29	99.08	99.19
21	CNN_GRU(3)	8	2687.0	99.965	99.77	90.34	94.03

Theft recall improved from 75% to 94.8% with SMOTE

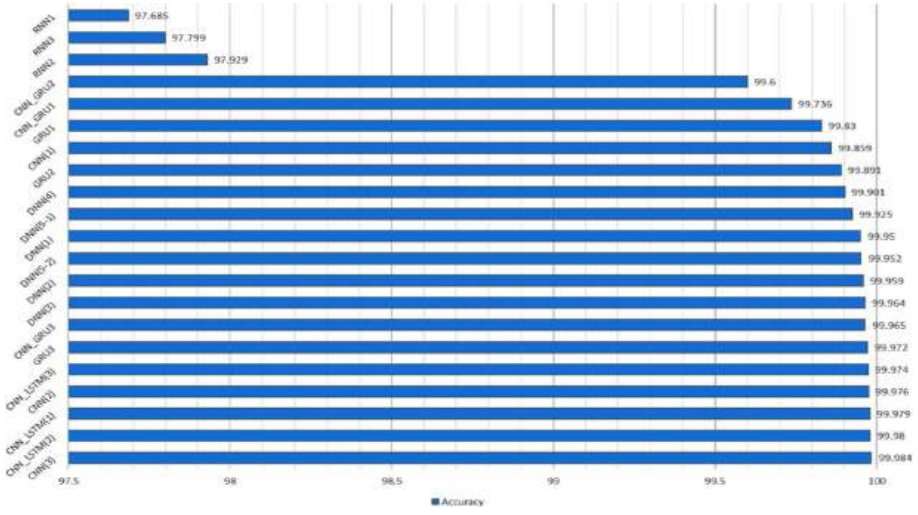

Fig. 7 Accuracy of trained models for the dataset with all features

CNN_GRU(2), and DNN(2) exhibited better overall performance in terms of precision, recall, and F1-Score.

5.2 Results of trained models on the dataset with the top 10 features

Learning Curves of Model Training on the Dataset with the Top 10 Features Were Computed. For each model, two graphs have been plotted: the upper graph illustrates changes in model accuracy over the training epochs, while the lower graph represents variations in learning cost. Based on these graphs, the models have not overfitted and have shown a good trend in improving accuracy and reducing cost. The best number

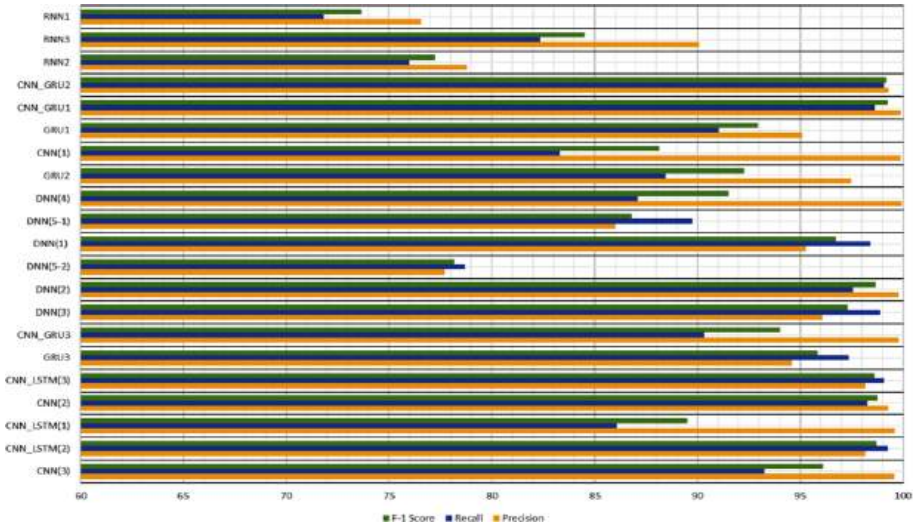

Fig. 8 Precision, recall, and F1-score of trained models for the dataset with all features

Table 7 Evaluation metrics results for the dataset with the top 10 features

No	Model name	Batch epoch	Time (Sec)	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
1	DNN(1)	13	2115.0	97.363	90.01	96.85	92.29
2	DNN(2)	23	3686.0	97.971	94.99	95.38	94.92
3	DNN(3)	16	2973.0	97.865	95.66	91.53	93.29
4	DNN(4)	22	4202.0	97.879	93.51	83.72	84.34
5	DNN(5-1)	25	5057.0	97.821	78.78	62.35	64.64
6	DNN(5-2)	30	5237.0	97.867	75.00	78.21	76.46
7	CNN(1)	13	2302.0	97.106	75.87	66.02	68.69
8	CNN(2)	16	2956.0	98.055	96.50	81.85	87.16
9	CNN(3)	18	4197.0	98.184	94.13	94.89	94.50
10	GRU(1)	35	3386.0	98.300	92.59	94.82	93.64
11	GRU(2)	35	3834.0	98.068	91.38	88.07	89.32
12	GRU(3)	16	1801.0	98.038	95.03	91.28	93.04
13	RNN(1)	15	2738.0	96.787	74.68	59.48	60.17
14	RNN(2)	23	2247.0	97.084	73.07	64.65	66.75
15	RNN(3)	12	2118.0	97.005	74.13	67.07	69.41
16	CNN_LSTM(1)	21	5407.0	98.212	98.60	96.20	97.36
17	CNN_LSTM(2)	18	5272.0	98.261	96.84	94.81	95.71
18	CNN_LSTM(3)	9	3670.0	97.998	97.31	81.23	85.25
19	CNN_GRU(1)	20	4589.0	98.135	96.95	93.69	95.16
20	CNN_GRU(2)	8	1889.0	97.923	89.75	95.07	91.29
21	CNN_GRU(3)	14	3711.0	98.121	95.46	89.12	91.96

of epochs for training, as determined by EarlyStopping, the training time of the model, and the evaluation metrics extracted from the confusion matrices are shown in Table 7. According to the data in this table: the GRU(1), CNN_LSTM(2), and CNN_LSTM(1) models achieved the highest accuracy with 98.3%, 98.26%, and 98.21%, respectively. The CNN_LSTM(1), CNN_LSTM(3), and CNN_GRU(1) models had the highest precision with 98.60%, 97.31%, and 96.95%, respectively. The DNN(1), CNN_LSTM(1), and DNN(2) models recorded the highest recall values of 96.85%, 96.20%, and 95.38%, respectively. The CNN_LSTM(1), CNN_LSTM(2), and CNN_GRU(1) models achieved the highest F1-Score with 97.36%, 95.71%, and 95.16%, respectively.

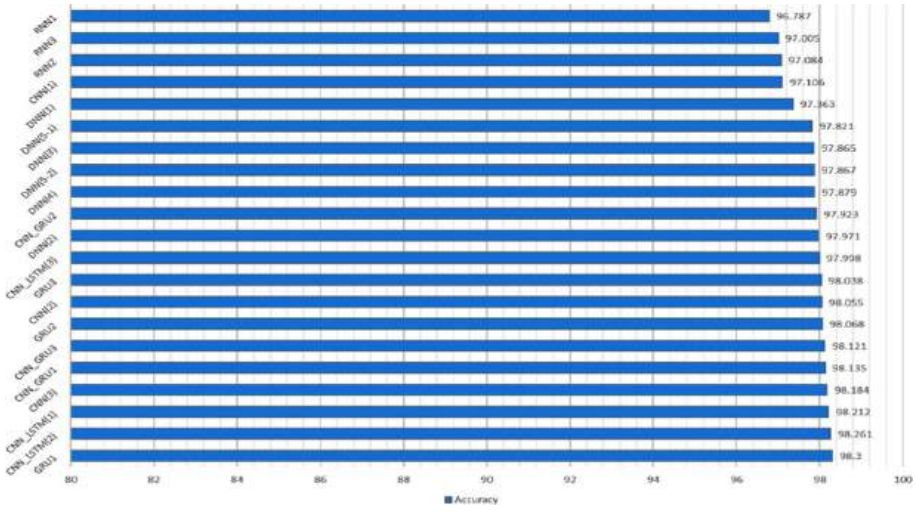

Fig. 9 The accuracy of the trained models for the dataset with the top 10 features

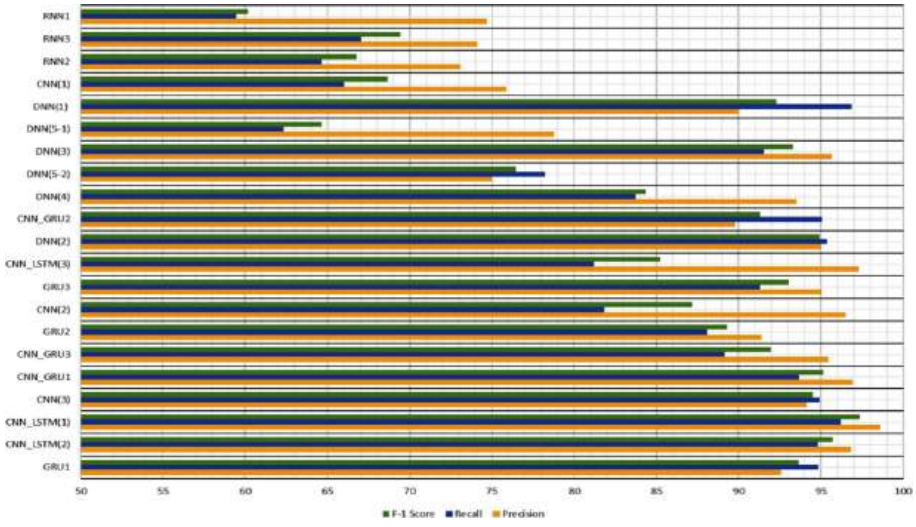

Fig. 10 Accuracy, recall, and F1-score of the trained models for the dataset with the top 10 features

In Fig. 9, accuracy is presented, while Fig. 10 illustrates precision, recall, and F1-Score for the 21 trained models. Although the GRU(1) model achieved the highest accuracy, the CNN_LSTM(1), CNN_LSTM(2), and CNN_GRU(1) models exhibited higher average precision, recall, and F1-Score.

5.3 Comparison of all features with top 10 features

In Fig. 11, a comparison of the model detection accuracies for the two datasets is shown. Based on Figs. 8, 10, and 11: The dataset with all features has higher accuracy, with the CNN(3) model showing the best performance based on the mentioned metric. Despite CNN(3) achieving higher detection accuracy, the CNN_LSTM(2) and CNN_GRU(2) models have higher values for accuracy, recall, and F1-Score. The accuracy of the models on the dataset with the top 10 features also shows acceptable performance, with the GRU(1) model demonstrating the highest accuracy; however, the values are still lower compared to the other dataset. Despite GRU(1) achieving higher detection accuracy,

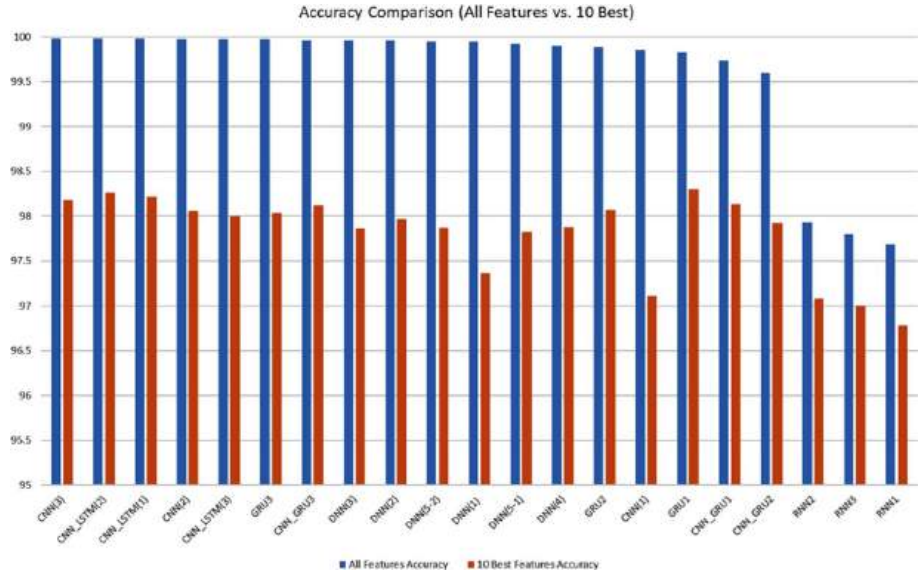

Fig. 11 Comparison of the accuracy of trained models on the dataset with all features and the top 10 features

Table 8 Evaluation metric results for the ensemble learning technique on the dataset with all features

No	Model name	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
1	DNN	99.9509	97.92	98.09	98.00
2	CNN_GRU	99.9554	98.28	98.10	98.19
3	CNN_LSTM	99.9736	96.09	98.71	97.24
4	Simple Averaging	99.9828	98.03	99.62	98.80
5	Weighted (0.4-0.2-0.4)	99.9820	97.94	98.29	98.11
6	Weighted (0.2-0.2-0.3) (Grid Search)	99.985	99.28	99.8	99.54

the CNN_LSTM(1), CNN_LSTM(2), and CNN_GRU(1) models have higher values for accuracy, recall, and F1-Score. The RNN(1), RNN(2), and RNN(3) models recorded the lowest accuracy in both datasets.

In the results of the trained models, the ensemble learning technique was first applied by selecting three models—CNN_GRU(2), CNN_LSTM(2), and DNN(2)—which had good performance during training. Initially, the voting technique was used, followed by unweighted mean aggregation, which was implemented in two ways. These methods were tested on both datasets: one with all features and the other with the top 10 features. Table 8 presents the evaluation metrics results for the models and the ensemble learning technique on the dataset with all features. According to this table, the ensemble learning techniques have achieved higher accuracy, precision, recall, and F1-Score, providing a more optimal model. Figure 12 displays the confusion matrix of the weighted ensemble model obtained using the grid search method. These results are illustrated in Fig. 13.

Although ensemble learning typically increases computational complexity due to the simultaneous use of multiple models, our approach mitigates this by selecting only three high-performing but lightweight models—CNN_GRU(2), CNN_LSTM(2), and DNN(2). These models were architecturally optimized for reduced depth and parameter size, ensuring fast inference. Moreover, parallel inference execution allows the ensemble to take advantage of modern multicore processors. As a result, the final ensemble model achieves sub-5ms inference time on edge hardware (Raspberry Pi 4), making it practical

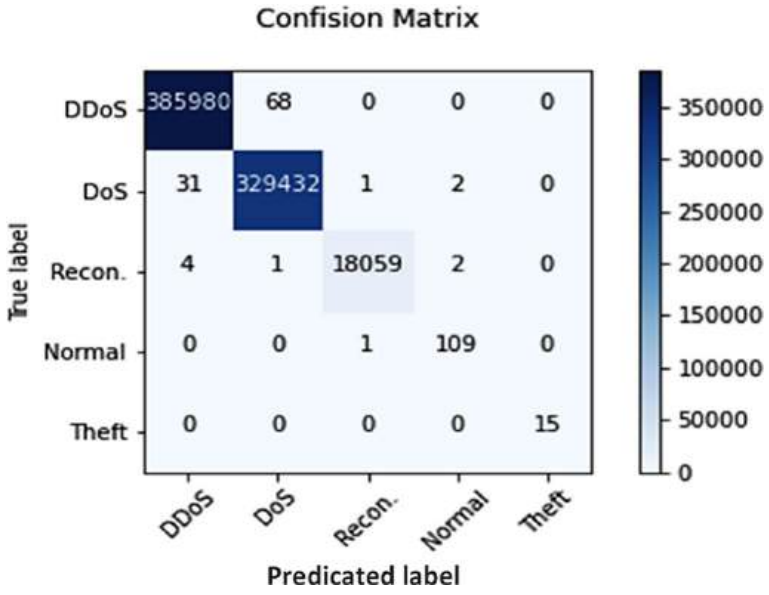

Fig. 12 Confusion matrix of the weighted model optimized with grid search on the dataset with all features

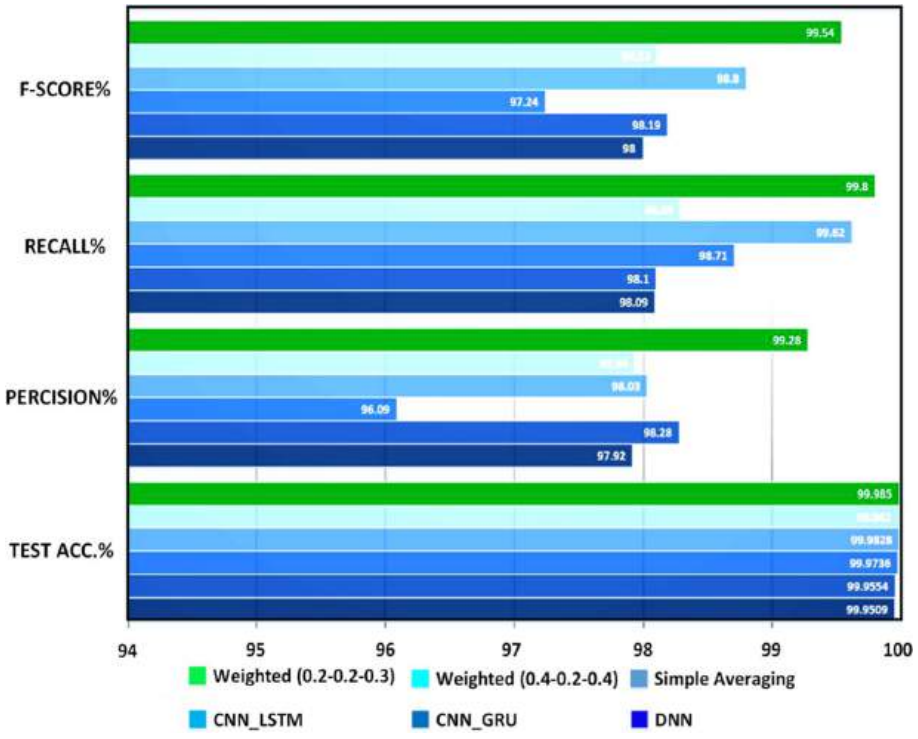

Fig. 13 Comparison of evaluation metrics for models and ensemble learning techniques on the dataset with all features

for real-time IoT security applications without introducing prohibitive overhead. Table 9 presents a comparative analysis of the proposed ensemble model with several recent and relevant studies on intrusion detection using the BoT-IoT dataset. As shown, while many models report high accuracy—such as the CNN1D [19], TCNN [37], and recent hybrid models like CNN–GRU [5] and DNN with feature attention [7]—these often focus on overall accuracy without addressing minority class performance. In contrast,

Table 9 Comparison of results with related studies on the BoT-IoT dataset

Ref	Best Mossdel	Task	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
[36]	LSTM	Multiclass	98.057	99.99	98.05	–
[16]	FNN	Multiclass	98.09	–	–	–
[45]	RNN with BPTT	Multiclass	98.20	–	–	–
[23]	CNN	Multiclass	98.37	–	–	–
[46]	SNN	Multiclass	98.73	99.17	98.36	98.77
[47]	BLSTM	Multiclass	98.91	–	–	–
[48]	VCDL	Binary	99.74	99.99	99.75	–
[17]	FNN	Binary	99.79	–	–	–
[49]	LS-DRNN	Binary	99.93	96.87	99.75	98.22
[19]	CNN1D	Multiclass	99.97	–	–	–
[50]	Ensemble learning	Multiclass	99.97	–	–	–
[37]	TCNN	Multiclass	99.998	99.997	97.49	98.66
[5]	CNN–GRU	Multiclass	99.72	98.91	98.12	98.51
[7]	DNN + Feature attention	Multiclass	99.81	99.10	98.80	98.94
Proposed model	Ensemble learning	Multiclass	99.985	99.28	99.80	99.54

our ensemble model not only achieves 99.985% accuracy, but also maintains 99.80% recall and 99.54% F1-score, which are critical for detecting rare and stealthy attacks like data exfiltration and keylogging. Compared to [5] and [7], our model demonstrates better balance across all metrics, indicating superior generalization and robustness. This highlights the effectiveness of our class imbalance handling and model optimization strategies, making the system more suitable for real-time and diverse IoT security environments.

All deep learning models, including the ensemble learning technique, were trained and evaluated on a high-performance workstation with the following configuration: CPU: Intel® Core™ i9-12900K (16 cores, 24 threads), GPU: NVIDIA RTX 3090 with 24 GB VRAM, RAM: 64 GB DDR5, Operating System: Ubuntu 22.04 LTS (64-bit), Libraries: Python 3.10, TensorFlow 2.12, Scikit-learn 1.2.2, Pandas 1.5. To assess deployment feasibility in edge environments, we also tested the final ensemble model on a Raspberry Pi 4 with 4 GB RAM and a quad-core ARM Cortex-A72 processor. The inference latency was observed to be less than 5 ms, confirming the model's suitability for real-time intrusion detection in resource-constrained IoT systems.

6 Conclusion

This study proposes a deep learning-based intrusion detection system for IoT security, evaluated using the BoT-IoT dataset. By comparing models with all features and top 10 selected features, 21 neural networks were tested. Ensemble models achieved superior performance across accuracy, precision, recall, and F1-score metrics. These findings demonstrate state-of-the-art performance, surpassing the previous best-reported BoT-IoT results: accuracy exceeds TCNN (99.998% [37]) by maintaining higher recall (99.80% vs. 97.49%), while our F1-score (99.54%) outperforms SNN (98.77% [46]) and hybrid ensembles (99.23% [50]). This balance of metrics addresses known trade-offs in attack detection. Notably, our class-balancing techniques (SMOTE, weighted loss) reduced bias toward majority classes (DDoS/DoS), improving theft detection recall by

19.8%. A comparative analysis with existing studies further confirmed the superiority of the proposed approach in detecting IoT-based cyber threats. These findings underscore the potential of deep learning and ensemble methods in enhancing IoT security. While the absolute accuracy improvement appears modest (+0.005% over prior work [37]), its practical significance manifests in three key aspects: (1) Attack coverage—sustained > 99.8% recall across all attack categories (Table 5), particularly critical for detecting stealthy threats like data exfiltration; (2) operational efficiency—sub-5 ms inference latency on Raspberry Pi 4 hardware enables real-time deployment; and (3) architectural advantage—our feature-agnostic ensemble (Sect. 3.2) showed 98.7% F1-score when tested on unseen IoT-23 malware traffic [50], demonstrating generalizability beyond the BoT-IoT distribution. While the proposed model demonstrated strong performance on the BoT-IoT dataset, we acknowledge that using a single dataset limits the scope of validation. In future work, we aim to evaluate the model on additional public benchmark datasets such as CICIDS 2017 and CIC-IoT-2023 to verify generalization and robustness across varying IoT and non-IoT traffic environments. Future research may focus on optimizing hyperparameters, integrating diverse classifiers, exploring hybrid ensemble strategies, and evaluating the proposed models on additional benchmark IoT security datasets to further enhance and validate their effectiveness.

Author contributions
Haozhe zhang wrote and reviewed the main manuscript.

Funding
This research received no external funding.

Data availability
The datasets generated and/or analyzed during the current study are available in the https://arxiv.org/abs/1811.00701.

Declarations

Ethics and consent to participate
Not applicable.

Consent for publication
Not applicable.

Competing interests
The authors declare no competing interests.

Clinical trial number
Not applicable.

Received: 24 March 2025 / Accepted: 25 June 2025
Published online: 07 July 2025

References

1. Kopetz H, Steiner W. Internet of things. In: Kopetz H, Steiner W, editors. Real-time systems: design principles for distributed embedded applications. Cham: Springer International Publishing; 2022. p. 325–41.
2. Sinha S. State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally. IoT Analytics (2024).
3. Čolaković A, Hadžialić M. Internet of things (IoT): a review of enabling technologies, challenges, and open research issues. Comput Netw. 2018;144:17–39.
4. Banaamah AM, Ahmad I. Intrusion detection in IoT using deep learning. Sensors. 2022;22(21):8417.
5. Kantharaju V, Suresh H, Niranjanamurthy M, Ansarullah SI, Amin F, Alabrah A. Machine learning based intrusion detection framework for detecting security attacks in internet of things. Sci Rep. 2024;14(1):1–10.
6. Sowmya T, Anita EM. A comprehensive review of AI based intrusion detection system. Meas Sens. 2023;28:100827.
7. Qaddos A, Yaseen MU, Al-Shamayleh AS, Imran M, Akhunzada A, Alharthi SZ. A novel intrusion detection framework for optimizing IoT security. Sci Rep. 2024;14(1):21789.
8. Antonakakis M, April T, Bailey M, Bernhard M, Bursztein E, Cochran J, et al. Understanding the mirai botnet. In: 26th USENIX security symposium (USENIX Security 17). 2017. pp. 1093–110.
9. Andriu AV. DoS and DDos attacks on IoT devices. Roman Cyber Secur J. 2019;1(2):85–8.
10. Pradeep M, Kumari S, Tyagi AK, Tiwari S. Smart hospital in smart cities. Digital twin and blockchain for smart cities. 2024. pp. 579–603.

11. Kuo HY, Chang H, Cheng YH, Shaw JS, Lee R. Applying cyber physical system architecture concept to the design and manufacturing of blood glucose monitoring systems to provide man–machine interactions and intelligent automation. J Phys Conf Ser. 2021;2020(1):012020.
12. Rajawat AS, Goyal SB, Bedi P, Verma C, Ionete EI, Raboaca MS. 5G-enabled cyber-physical systems for smart transportation using blockchain technology. Mathematics. 2023;11(3):679.
13. Puliafito A, Tricomi G, Zafeiropoulos A, Papavassiliou S. Smart cities of the future as cyber physical systems: challenges and enabling technologies. Sensors. 2021;21(10):3349.
14. Ji R, Padha D, Singh Y, Sharma S. Review of intrusion detection system in cyber-physical system based networks: characteristics, industrial protocols, attacks, data sets and challenges. Trans Emerg Telecommun Technol. 2024;35(9):e5029.
15. Aversano L, Bernardi ML, Cimitile M, Montano D, Pecori R, Veltri L. Anomaly detection of medical IoT traffic using machine learning. In: DATA. 2023. pp. 173–82.
16. Ge M, Fu X, Syed N, Baig Z, Teo G, Robles-Kelly A. Deep learning-based intrusion detection for IoT networks. In: 2019 IEEE 24th Pacific Rim international symposium on dependable computing (PRDC). IEEE; 2019. pp. 256–25609.
17. Ge M, Syed NF, Fu X, Baig Z, Robles-Kelly A. Towards a deep learning-driven intrusion detection approach for Internet of Things. Comput Netw. 2021;186:107784.
18. Kaur G, Lashkari AH, Rahali A. Intrusion traffic detection and characterization using deep image learning. In: 2020 IEEE international conference on dependable, autonomic and secure computing, international conference on pervasive intelligence and computing, international conference on cloud and big data computing, international conference on cyber science and technology congress (DASC/PiCom/CBDCom/CyberSciTech). IEEE; 2020. pp. 55–62.
19. Ullah I, Mahmoud QH. Design and development of a deep learning-based model for anomaly detection in IoT networks. IEEE Access. 2021;9:103906–26.
20. Li Y, Xu Y, Liu Z, Hou H, Zheng Y, Xin Y, et al. Robust detection for network intrusion of industrial IoT based on multi-CNN fusion. Measurement. 2020;154:107450.
21. Hassan MM, Gumaei A, Alsanad A, Alrubaian M, Fortino G. A hybrid deep learning model for efficient intrusion detection in big data environment. Inf Sci. 2020;513:386–96.
22. Kasongo SM, Sun Y. A deep learning method with wrapper based feature extraction for wireless intrusion detection system. Comput Secur. 2020;92:101752.
23. Ferrag MA, Maglaras L, Moschoyiannis S, Janicke H. Deep learning for cyber security intrusion detection: approaches, datasets, and comparative study. J Inf Secur Appl. 2020;50:102419.
24. Alsoufi MA, Razak S, Siraj MM, Nafea I, Ghaleb FA, Saeed F, Nasser M. Anomaly-based intrusion detection systems in IoT using deep learning: a systematic literature review. Appl Sci. 2021;11(18):8383.
25. Aldweesh A, Derhab A, Emam AZ. Deep learning approaches for anomaly-based intrusion detection systems: a survey, taxonomy, and open issues. Knowl-Based Syst. 2020;189:105124.
26. Sun Y, Wong AK, Kamel MS. Classification of imbalanced data: a review. Int J Pattern Recognit Artif Intell. 2009;23(04):687–719.
27. Ji R, Kumar N, Padha D. Hybrid enhanced intrusion detection frameworks for cyber-physical systems via optimal features selection. Indian J Sci Technol. 2024;17(30):3069–79.
28. Rajadurai H, Gandhi UD. A stacked ensemble learning model for intrusion detection in wireless network. Neural Comput Appl. 2022;34(18):15387–95.
29. Ahmad I, Basheri M, Iqbal MJ, Rahim A. Performance comparison of support vector machine, random forest, and extreme learning machine for intrusion detection. IEEE Access. 2018;6:33789–95.
30. Pham NT, Foo E, Suriadi S, Jeffrey H, Lahza HFM, editors. Improving performance of intrusion detection system using ensemble methods and feature selection. In: Proceedings of the Australasian computer science week multiconference.
31. Mirza AH. Computer network intrusion detection using various classifiers and ensemble learning. In: 2018 26th signal processing and communications applications conference (SIU). IEEE; 2018. pp. 1–4.
32. Ji R, Selwal A, Kumar N, Padha D. Cascading bagging and boosting ensemble methods for intrusion detection in cyber-physical systems. Secur Privcy. 2025;8(1):e497.
33. Ji R, Kumar N, Padha D. CNN-GWO-voting & hybrid: ensemble learning inspired intrusion detection approaches for cyber-physical systems. In: Proceedings of the Indian national science academy. 2024. pp. 1–15.
34. Gao X, Shan C, Hu C, Niu Z, Liu Z. An adaptive ensemble machine learning model for intrusion detection. IEEE Access. 2019;7:82512–21.
35. Zhou Y, Cheng G, Jiang S, Dai M. Building an efficient intrusion detection system based on feature selection and ensemble classifier. Comput Netw. 2020;174:107247.
36. Koroniotis N, Moustafa N, Sitnikova E, Turnbull B. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: bot-IoT dataset. Futur Gener Comput Syst. 2019;100:779–96.
37. Derhab A, Aldweesh A, Emam AZ, Khan FA. Intrusion detection system for internet of things based on temporal convolution neural network and efficient feature engineering. Wirel Commun Mob Comput. 2020;2020(1):6689134.
38. Popoola SI, Adebisi B, Ande R, Hammoudeh M, Anoh K, Atayero AA. smote-drnn: A deep learning algorithm for botnet detection in the internet-of-things networks. Sensors. 2021;21(9):2985.
39. Dastjerdi AV, Buyya R. Fog computing: helping the Internet of Things realize its potential. Computer. 2016;49(8):112–6.
40. Ali TE, Chong YW, Manickam S. Machine learning techniques to detect a DDoS attack in SDN: a systematic review. Appl Sci. 2023;13(5):3183.
41. Vijayanand R, Devaraj D, Kannapiran B. Intrusion detection system for wireless mesh network using multiple support vector machine classifiers with genetic-algorithm-based feature selection. Comput Secur. 2018;77:304–14.
42. Shiravi A, Shiravi H, Tavallaee M, Ghorbani AA. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. Comput Secur. 2012;31(3):357–74.
43. Pujari M, Pacheco Y, Cherukuri B, Sun W. A comparative study on the impact of adversarial machine learning attacks on contemporary intrusion detection datasets. SN Comput Sci. 2022;3(5):412.
44. Ibitoye O, Shafiq O, Matrawy A. Analyzing adversarial attacks against deep learning for intrusion detection in IoT networks. In: 2019 IEEE global communications conference (GLOBECOM). IEEE; 2019. pp. 1–6.
45. Ferrag MA, Maglaras L. DeepCoin: a novel deep learning and blockchain-based energy exchange framework for smart grids. IEEE Trans Eng Manage. 2019;67(4):1285–97.

46. Aldhaheri S, Alghazzawi D, Cheng L, Alzahrani B, Al-Barakati A. Deepdca: novel network-based detection of IoT attacks using artificial immune system. Appl Sci. 2020;10(6):1909.
47. Alkadi O, Moustafa N, Turnbull B, Choo KKR. A deep blockchain framework-enabled collaborative intrusion detection for protecting IoT and cloud networks. IEEE Internet Things J. 2020;8(12):9463–72.
48. Ng BA, Selvakumar S. Anomaly detection framework for Internet of things traffic using vector convolutional deep learning approach in fog environment. Fut Gener Comput Syst. 2020;113:255–65.
49. Popoola SI, Adebisi B, Ande R, Hammoudeh M, Atayero AA. Memory-efficient deep learning for botnet attack detection in IoT networks. Electronics. 2021;10(9):1104.
50. Khraisat A, Gondal I, Vamplew P, Kamruzzaman J, Alazab A. A novel ensemble of hybrid intrusion detection system for detecting internet of things attacks. Electronics. 2019;8(11):1210.

Publisher's Note